LAPORAN UTS
PEMROGRAMAN BERORIENTASI OBJEK (PBO)
KELAS A

Dosen Pengampu :

Himawan Wicaksono, S.ST.,M.T.

Disusun oleh :

Muhammad Syahrul Nugroho 04231064

Link (URL) ke Repository GitHub Publik:
https://github.com/04231064-SYAHRUL/UTS_PBO_M-SYAHRUL-NUGROHO_KOS-HUB_A

PROGRAM STUDI TEKNIK ELEKTRO

JURUSAN TEKNIK ELEKTRO INFORMATIKA DAN BISNIS

FAKULTAS SAINS DAN TEKNOLOGI INFORMASI

INSTITUT TEKNOLOGI KALIMANTAN

2026

TAHAP I

Analisis SDLC & Manajemen Proyek

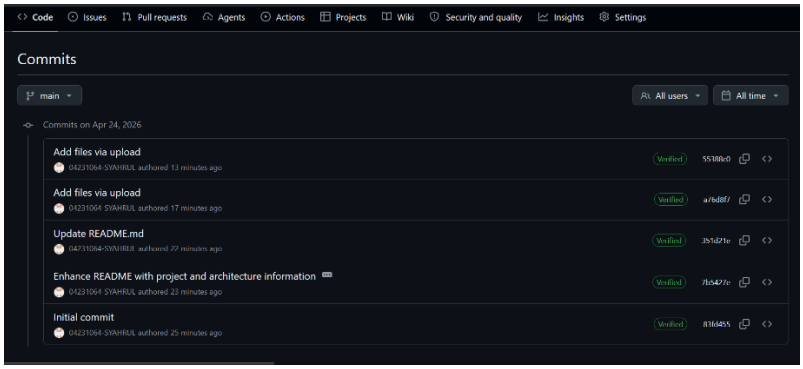
1.1 Link (URL) Repository GitHub

Sebagai bentuk transparansi dan pengelolaan kode sumber yang terstruktur, seluruh pengembangan sistem **Kost-Hub** telah didokumentasikan dan diunggah ke dalam repositori GitHub publik. Repositori ini berfungsi sebagai wadah penyimpanan *version control* yang memungkinkan pelacakan riwayat perubahan kode secara kronologis. Repositori dapat diakses melalui tautan berikut:

https://github.com/04231064-SYAHRUL/UTS_PBO_M-SYAHRUL-NUGROHO_KOS-HUB_A

1.2 Bukti Versi Kontrol (Version Control)

Pengembangan sistem Kost-Hub dilakukan dengan memanfaatkan fitur *commit* pada Git untuk mendokumentasikan setiap kemajuan fitur secara berkala. Melalui riwayat *commit* ini, dapat dilihat proses evolusi perangkat lunak, mulai dari inisialisasi struktur dasar, implementasi logika enkapsulasi pada kelas-kelas entitas, hingga tahap finalisasi skenario simulasi. Dokumentasi ini menjadi bukti bahwa sistem dibangun secara sistematis dan bukan merupakan hasil pengerjaan sekali jadi.



Gambar 1.2.2 Screenshot Github 5 Commits

1.2 Perbandingan Paradigma OOP vs Prosedural

Dalam sistem manajemen kos, data seperti saldo penyewa dan status kamar adalah aset yang sangat krusial. Dalam paradigma prosedural, data ini cenderung bersifat terbuka dan rentan dimanipulasi secara tidak sengaja oleh fungsi manapun karena tidak adanya batasan akses yang jelas.

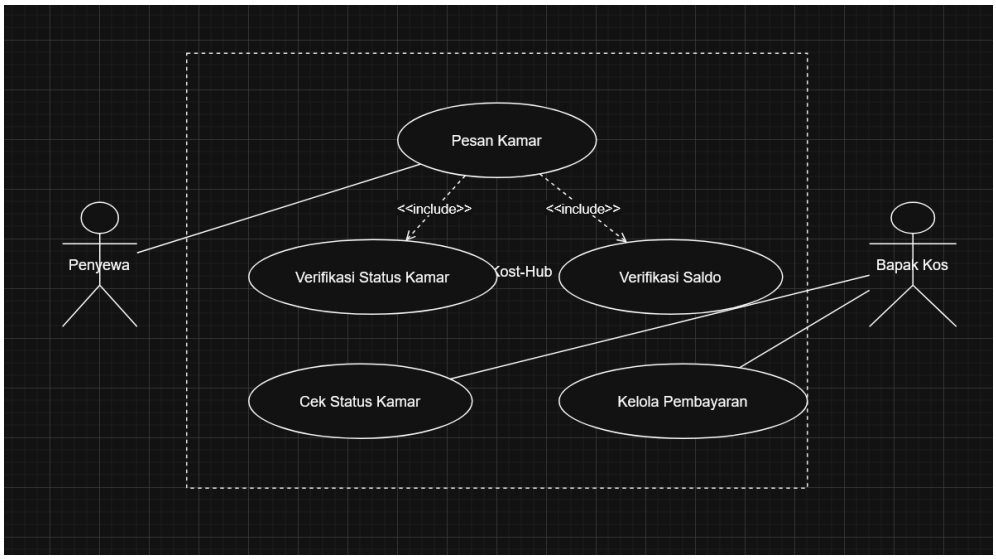
Sebaliknya, paradigma **Object-Oriented Programming (OOP)** menyelesaikan masalah tersebut melalui prinsip **Enkapsulasi**. Dengan menggunakan modifier private set, data sensitif hanya bisa diubah melalui jalur resmi yang memiliki validasi logika, sehingga integritas keuangan dan operasional kos tetap terjaga dan aman dari kesalahan akses. Paradigma **Pemrograman Berorientasi Objek (OOP)** hadir dengan solusi berupa prinsip **Enkapsulasi**. Konsep ini memungkinkan penyatuan antara data dan metode yang mengelolanya ke dalam sebuah unit mandiri yang disebut Objek. Dalam kode sistem **Kost-Hub**, perlindungan data diterapkan secara ketat menggunakan modifier akses private set. Mekanisme ini memastikan adanya *Data Hiding*, di mana status internal sebuah objek tidak dapat diintervensi langsung dari luar kelas, melainkan harus melalui "gerbang" validasi resmi (metode publik).

Secara teknis, penggunaan OOP memastikan bahwa saldo penyewa hanya akan berkurang apabila fungsi pembayaran mengembalikan nilai *true* setelah memeriksa kecukupan dana. Begitu pula dengan status kamar yang hanya akan berubah jika logika pengecekan ketersediaan terpenuhi. Dengan demikian, sistem **Kost-Hub** memiliki tingkat ketahanan data yang tinggi, lebih terstruktur, dan jauh lebih aman dibandingkan sistem yang dibangun dengan pendekatan prosedural tradisional.

TAHAP II

PEMODELAN SISTEM

2.1 Use Case Diagram



Gambar 2.1 Diagram Sistem Use Case Kost-Hub

Dalam perancangan sistem **Kost-Hub**, interaksi antara pengguna dan sistem dimodelkan melalui *Use Case Diagram* yang melibatkan dua aktor utama, yaitu **Penyewa** dan **BapakKos**. Penyewa memiliki peran aktif dalam operasional harian, di mana mereka dapat melakukan proses **Pemesanan Kamar** dan **Pembayaran Tagihan**. Di sisi lain, BapakKos berfungsi sebagai manajer sistem yang memantau **Pendapatan** dan memastikan ketersediaan fasilitas. Hubungan antara proses pembayaran dan validasi internal digambarkan melalui relasi *include*, yang berarti setiap kali penyewa membayar tagihan, sistem secara otomatis akan menjalankan fungsi **Verifikasi Saldo** dan **Pengecekan Status Kamar**. Hal ini memastikan bahwa transaksi hanya terjadi jika data di lapangan (status kamar) dan data finansial (saldo) berada dalam kondisi yang valid.

Sistem terdiri dari kelas-kelas utama dengan tanggung jawab yang terpisah (*separation of concerns*):

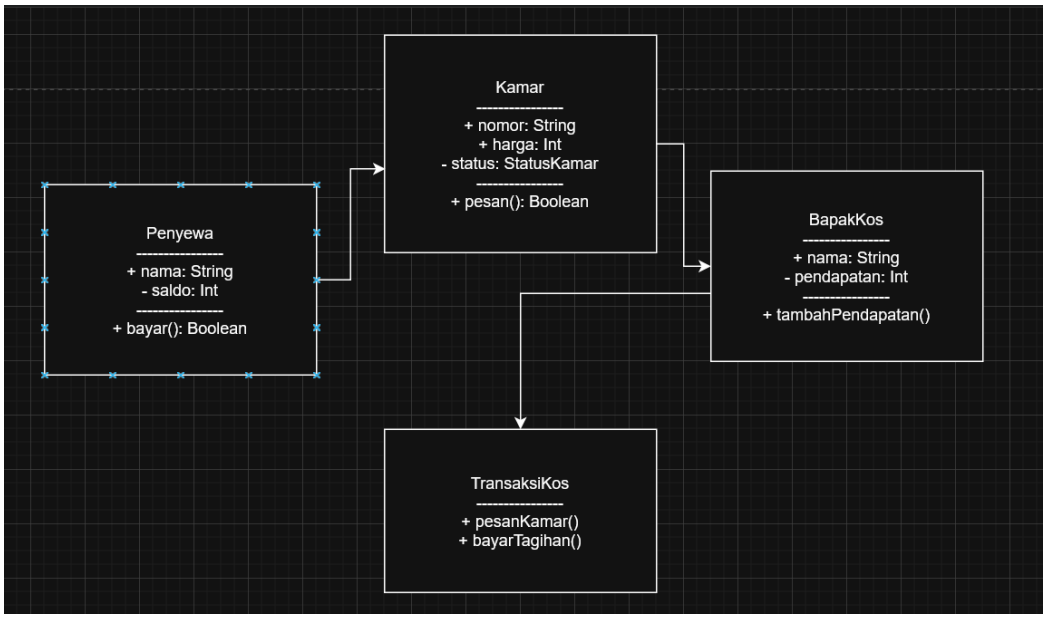
1. **Penyewa**: Memiliki atribut nama (String) dan saldo (Int, privat). Metode bayar() bertugas sebagai jalur resmi pengurangan saldo.

2. **Kamar**: Berperan sebagai objek yang disewa dengan atribut nomor, harga, dan status

(KOSONG/TERISI) yang dilindungi aksesnya.

3. **BapakKos**: Memiliki atribut pendapatan (Int, privat) yang hanya bisa bertambah melalui metode resmi.
4. **TransaksiKos**: Kelas pusat logika bisnis yang mengintegrasikan interaksi antar objek (uses Penyewa, Kamar, dan BapakKos)

2.2 Class Diagram



Gambar 2.2 Diagram Sistem Clas Diagram Kost-Hub

Struktur logika sistem **Kost-Hub** dimodelkan melalui *Class Diagram* yang menitikberatkan pada keamanan data melalui prinsip enkapsulasi. Sistem ini dibangun di atas empat komponen utama yang saling berinteraksi secara mandiri namun tetap terintegrasi:

Pertama, kelas **Penyewa** berfungsi sebagai subjek utama dalam sistem. Kelas ini menyimpan atribut nama dan saldo. Keamanan data saldo menjadi prioritas utama dengan penerapan akses private set, yang berarti nilai saldo hanya dapat dimodifikasi melalui metode internal bayar(). Metode ini bertindak sebagai filter validasi untuk memastikan bahwa transaksi hanya terjadi jika saldo mencukupi, sehingga mencegah terjadinya nilai saldo negatif atau manipulasi data finansial secara ilegal dari luar kelas.

Kedua, terdapat kelas **Kamar** yang merepresentasikan objek fisik dalam bisnis kos. Atribut penting dalam kelas ini mencakup nomor kamar, harga sewa, dan status ketersediaan yang menggunakan tipe data enum (**StatusKamar**). Status ini sangat krusial dalam aturan bisnis; metode pesan() dirancang agar perubahan status dari KOSONG menjadi TERISI hanya bisa dilakukan jika kamar tersebut benar-benar belum memiliki penghuni. Hal ini secara otomatis mencegah terjadinya kesalahan operasional seperti pemesanan ganda (*overbooking*).

Ketiga, kelas **BapakKos** berperan sebagai entitas pengelola dan penerima pendapatan. Atribut pendapatannya juga dilindungi secara privat dan hanya dapat diperbarui melalui metode resmi tambahPendapatan(). Terakhir, seluruh interaksi antar kelas ini diorkestrasi oleh kelas **TransaksiKos**. Kelas ini berfungsi sebagai *Controller* yang menghubungkan objek Penyewa, Kamar, dan BapakKos. Di sinilah alur logika utama dieksekusi: pemesanan akan dibatalkan jika kamar berstatus "Terisi", dan transaksi pembayaran hanya akan sukses jika penyewa memiliki saldo yang cukup, yang kemudian secara otomatis akan menambah saldo pendapatan pemilik kos.

Catatan:

- **Paragraf 1** membahas Kelas Penyewa (Enkapsulasi Saldo).
- **Paragraf 2** membahas Kelas Kamar (Logika Status & Enum).
- **Paragraf 3** membahas Kelas BapakKos dan Kelas Transaksi (Logika Bisnis/Aturan Bisnis).

TAHAP III

IMPLEMENTASI ENKAPSULASI KOTLIN

3.1 Codingan Kotlin

```
// ============================================================
//        UJIAN TENGAH SEMESTER - PEMROGRAMAN BERORIENTASI OBJEK
// ============================================================

enum class StatusKamar {
    KOSONG, TERISI
}

class Penyewa(
    val nama: String,
    saldoAwal: Int
) {
    var saldo: Int = saldoAwal
        private set

    fun bayar(jumlah: Int): Boolean {
        return if (jumlah <= saldo) {
            saldo -= jumlah
            true
        } else {
            false
        }
    }
}

class BapakKos(
    val nama: String
) {
    var pendapatan: Int = 0
        private set

    fun tambahPendapatan(jumlah: Int) {
        pendapatan += jumlah
    }
}

class Kamar(
    val nomor: String,
    val harga: Int
) {
    var status: StatusKamar = StatusKamar.KOSONG
```

```kotlin
class Kamar(
    val nomor: String,
    val harga: Int
) {
    var status: StatusKamar = StatusKamar.KOSONG
        private set

    fun pesan(): Boolean {
        return if (status == StatusKamar.KOSONG) {
            status = StatusKamar.TERISI
            true
        } else {
            false
        }
    }
}

class TransaksiKos {

    fun pesanKamar(penyewa: Penyewa, kamar: Kamar) {

        println("\n┌──────────────────────────────┐")
        println("│      PERMINTAAN PEMESANAN      │")
        println("└──────────────────────────────┘")
        println("  Penyewa : ${penyewa.nama}")
        println("  Kamar   : ${kamar.nomor}")
        println("  Status  : ${kamar.status}")

        if (!kamar.pesan()) {
            println("\n  [DITOLAK] Kamar sudah terisi!")
            return
        }

        println("\n  [BERHASIL] Kamar berhasil dipesan!")
        println("  Status sekarang : ${kamar.status}")
    }

    fun bayarTagihan(penyewa: Penyewa, bapakKos: BapakKos, kamar: Kamar) {

        val harga = kamar.harga
```

```
fun main() {

    println("╔═══════════════════════════════════════╗")
    println("║      SIMULASI UTS - PBO KOST-HUB      ║")
    println("╚═══════════════════════════════════════╝")
    println("╔═══════════════════════════════════════╗")
    println("║  Tema : Manajemen Kos Mahasiswa       ║")
    println("╚═══════════════════════════════════════╝")

    val transaksi = TransaksiKos()
    val bapakKos = BapakKos("Tarjo")

    // ── SKENARIO 1: GAGAL
    println("\n═══════════════════════════════════════")
    println("  SKENARIO 1 — GAGAL")
    println("═══════════════════════════════════════")

    val penyewa1 = Penyewa("Syahrul", 300000)
    val kamar1 = Kamar("A1", 500000)

    transaksi.pesanKamar(penyewa1, kamar1)
    transaksi.pesanKamar(penyewa1, kamar1)
    transaksi.bayarTagihan(penyewa1, bapakKos, kamar1)

    // ── SKENARIO 2: SUKSES
    println("\n═══════════════════════════════════════")
    println("  SKENARIO 2 — SUKSES")
    println("═══════════════════════════════════════")

    val penyewa2 = Penyewa("Gery", 1000000)
    val kamar2 = Kamar("B1", 400000)

    transaksi.pesanKamar(penyewa2, kamar2)
    transaksi.bayarTagihan(penyewa2, bapakKos, kamar2)

    println("\n╔═══════════════════════════════════╗")
    println("║          SIMULASI SELESAI           ║")
    println("╚═══════════════════════════════════╝")
}
```

3.2 ANALISIS STRUKTUR KODE (STEP-BY-STEP)

1. Definisi Status dengan Enum Class

```
enum class StatusKamar {
    KOSONG, TERISI
}
```

Penjelasan: Bagian ini mendefinisikan tipe data khusus bernama StatusKamar. Penggunaan enum memastikan bahwa status kamar hanya bisa berisi dua nilai pasti: KOSONG atau TERISI. Ini mencegah adanya status ilegal (seperti salah ketik) yang bisa merusak logika program , Program diawali dengan pendefinisian **StatusKamar** menggunakan *enum class*. Penggunaan *enum* ini sangat krusial untuk menjaga integritas data karena memastikan bahwa status kamar hanya memiliki dua nilai pasti, yaitu KOSONG atau TERISI. Hal ini mencegah masuknya data ilegal atau kesalahan pengetikan (seperti "kosung" atau "full") yang dapat mengganggu jalannya logika sistem pada tahap-tahap berikutnya.

2. Entitas Penyewa (Data Pelanggan & Saldo)

```
class Penyewa(val nama: String, saldoAwal: Int) {
    var saldo: Int = saldoAwal
        private set

    fun bayar(jumlah: Int): Boolean {
        return if (jumlah <= saldo) {
            saldo -= jumlah
            true
        } else false
    }
}
```

Pada kelas **Penyewa**, diterapkan prinsip enkapsulasi pada atribut saldo dengan menggunakan modifier private set. Struktur ini memungkinkan saldo dibaca oleh objek lain, namun perubahannya hanya bisa dilakukan melalui fungsi internal bayar(). Fungsi ini berperan sebagai "penjaga pintu" yang melakukan validasi otomatis untuk memeriksa apakah saldo mencukupi sebelum transaksi dilakukan, sehingga mencegah nilai saldo menjadi negatif.

- **Enkapsulasi:** Variabel saldo menggunakan private set. Artinya, pihak luar bisa melihat jumlah saldo, tetapi tidak bisa mengubahnya secara langsung (misal: penyewa.saldo = 1000 akan error).
- **Logika Bisnis:** Saldo hanya bisa berkurang melalui fungsi bayar(). Fungsi ini memiliki "satpam" berupa percabangan if yang mengecek apakah saldo cukup. Jika cukup, saldo dikurangi dan mengembalikan nilai true.

3. Entitas Kamar (Objek Sewa)

```
class Kamar(val nomor: String, val harga: Int) {
    var status: StatusKamar = StatusKamar.KOSONG
        private set

    fun pesan(): Boolean {
        return if (status == StatusKamar.KOSONG) {
            status = StatusKamar.TERISI
            true
        } else false
    }
}
```

- **Logika Validasi:** Sama seperti saldo, status kamar diproteksi dengan private set.
- **Aturan Bisnis:** Fungsi pesan() adalah implementasi dari aturan: *"Pemesanan gagal jika kamar terisi"*. Fungsi ini mengecek status; jika masih KOSONG, maka status diubah jadi TERISI. Jika sudah terisi, permintaan ditolak (false).

Kelas **Kamar** merepresentasikan objek fisik yang memiliki harga dan nomor unik. Serupa dengan kelas penyewa, atribut status diproteksi untuk memastikan perubahan hanya terjadi melalui fungsi pesan(). Fungsi tersebut mengimplementasikan aturan bisnis utama, yaitu hanya mengizinkan pemesanan jika kamar masih dalam status KOSONG. Jika kamar sudah terisi, sistem secara otomatis menolak permintaan, sehingga menjamin tidak terjadi pemesanan ganda (*overbooking*).

4. Entitas BapakKos (Manajemen Pendapatan)

```
class BapakKos(val nama: String) {
    var pendapatan: Int = 0
        private set

    fun tambahPendapatan(jumlah: Int) {
        pendapatan += jumlah
    }
}
```

Kelas **BapakKos** berfungsi sebagai entitas penerima manfaat. Atribut pendapatan dilindungi agar tidak dapat dimanipulasi secara langsung. Akumulasi pendapatan hanya dapat bertambah jika dipicu oleh transaksi yang sukses, memastikan catatan keuangan pemilik kos tetap akurat dan sinkron dengan pengeluaran penyewa.

5. Controller: TransaksiKos & Simulasi Main

```
class TransaksiKos {
    fun pesanKamar(penyewa: Penyewa, kamar: Kamar) { ... }
    fun bayarTagihan(penyewa: Penyewa, bapakKos: BapakKos, kamar: Kamar) { ... }
}
```

Sebagai pusat kendali, kelas **TransaksiKos** bertindak sebagai perantara yang menghubungkan ketiga entitas sebelumnya. Di dalam kelas ini, alur pembayaran tagihan diproses dengan melakukan pengecekan berlapis: memastikan kamar sudah dipesan dan memastikan saldo penyewa cukup. Seluruh rangkaian logika ini diuji pada fungsi **main()** melalui skenario gagal (saldo kurang/kamar penuh) dan skenario sukses, guna membuktikan bahwa sistem bekerja secara konsisten sesuai aturan manajemen kos

```
}
```

3.3 Output Codingan Kotlin

```
╔═══════════════════════════════════════╗
║     SIMULASI UTS - PBO KOST-HUB       ╕
╞═══════════════════════════════════════╡
│  Tema : Manajemen Kos Mahasiswa       │
╘═══════════════════════════════════════╛

═══════════════════════════════════════
  SKENARIO 1 — GAGAL
═══════════════════════════════════════

┌─────────────────────────────────┐
│      PERMINTAAN PEMESANAN       │
└─────────────────────────────────┘
  Penyewa : Syahrul
  Kamar   : A1
  Status  : KOSONG

  [BERHASIL] Kamar berhasil dipesan!
  Status sekarang : TERISI
```

```
┌──────────────────────────────────────┐
│        PERMINTAAN PEMESANAN          │
└──────────────────────────────────────┘

 Penyewa : Syahrul
 Kamar   : A1
 Status  : TERISI

 [DITOLAK] Kamar sudah terisi!


┌──────────────────────────────────────┐
│        PEMBAYARAN TAGIHAN            │
└──────────────────────────────────────┘

 Penyewa : Syahrul
 Harga   : Rp500000
 Saldo   : Rp300000

 [DITOLAK] Saldo tidak mencukupi!
 Dibutuhkan: Rp500000 | Saldo: Rp300000

═══════════════════════════════════════════
 SKENARIO 2 — SUKSES
═══════════════════════════════════════════


┌──────────────────────────────────────┐
│        PERMINTAAN PEMESANAN          │
└──────────────────────────────────────┘

 Penyewa : Gery
 Kamar   : B1
 Status  : KOSONG
```

```
[BERHASIL] Kamar berhasil dipesan!
Status sekarang : TERISI

PEMBAYARAN TAGIHAN

Penyewa : Gery
Harga   : Rp400000
Saldo   : Rp1000000

[BERHASIL] Pembayaran sukses!
Sisa saldo     : Rp600000
Pendapatan kos : Rp400000

SIMULASI SELESAI
```

Penjelasan Output Per Skenario

Analisis Output Per Skenario

Analisis Skenario 1 — GAGAL (Penyewa: Syahrul)

Skenario pertama mendemonstrasikan bagaimana sistem mempertahankan integritas data ketika dihadapkan pada kondisi yang melanggar aturan bisnis.

- Validasi Status Kamar: Saat Syahrul melakukan pemesanan pertama pada kamar A1, status berhasil berubah menjadi TERISI. Namun, saat perintah pemesanan kedua dijalankan pada objek kamar yang sama, sistem memberikan output [DITOLAK] Kamar sudah terisi!. Ini membuktikan fungsi pesan() pada kelas Kamar bekerja secara konsisten untuk mencegah tumpang tindih penyewa pada satu kamar yang sama.
- Validasi Finansial: Pada tahap pembayaran, sistem menampilkan bahwa harga kamar adalah Rp500.000 sedangkan saldo Syahrul hanya Rp300.000. Output menunjukkan [DITOLAK] Saldo tidak mencukupi!. Hal ini memverifikasi bahwa metode bayar() pada kelas Penyewa berhasil melakukan pengecekan logika sebelum mengurangi saldo.
- Kesimpulan Logika: Meskipun kamar sempat dipesan, kegagalan transaksi pembayaran mengakibatkan pendapatan Bapak Kos tetap tidak bertambah (tetap Rp0 dari transaksi Syahrul). Ini menunjukkan koordinasi yang baik antar objek untuk menjaga validitas data finansial.

Analisis Skenario 2 — SUKSES (Penyewa: Gery)

Skenario kedua menunjukkan alur kerja ideal di mana semua persyaratan (status kamar tersedia dan saldo mencukupi) terpenuhi.

- Kelancaran Alur Kerja: Gery melakukan pemesanan pada kamar B1 yang berstatus KOSONG. Sistem segera merespons dengan [BERHASIL] Kamar berhasil dipesan! dan memperbarui status secara *real-time* menjadi TERISI.
- Keberhasilan Transaksi: Saat pembayaran dilakukan, saldo Gery sebesar Rp1.000.000 divalidasi terhadap harga kamar Rp400.000. Karena syarat terpenuhi, sistem mengeluarkan output [BERHASIL] Pembayaran sukses!

- Integritas Data Akhir: Output secara otomatis menampilkan Sisa saldo: Rp600.000 dan Pendapatan kos: Rp400.000. Angka-angka ini membuktikan secara matematis bahwa logika pengurangan saldo penyewa dan penambahan pendapatan pemilik kos telah terhubung secara akurat di dalam kelas TransaksiKos.

Kesimpulan Output

Secara kolektif, kedua skenario ini membuktikan bahwa program Kost-Hub bukan sekadar menampilkan teks, melainkan benar-benar menjalankan prinsip Enkapsulasi PBO. Setiap pesan [DITOLAK] adalah bukti bahwa "pintu masuk" data telah dijaga ketat oleh logika validasi, sementara setiap angka yang diperbarui pada skenario [BERHASIL] membuktikan bahwa aliran data antar objek (Penyewa, Kamar, BapakKos) telah tersinkronisasi dengan sempurna sesuai aturan bisnis manajemen kos.

LAMPIRAN

- *Screenshot Codingan Kotlin*

Kotlin

Solutions Case studies Docs API Community Teach Play

2.3.21 JVM Program arguments Copy link Share code Run

```kotlin
// ==========================================================
//       UJIAN TENGAH SEMESTER - PEMROGRAMAN BERORIENTASI OBJEK
// ==========================================================

enum class StatusKamar {
    KOSONG, TERISI
}

class Penyewa(
    val nama: String,
    saldoAwal: Int
) {
    var saldo: Int = saldoAwal
        private set

    fun bayar(jumlah: Int): Boolean {
        return if (jumlah <= saldo) {
            saldo -= jumlah
            true
        } else {
            false
        }
    }
}

class BapakKos(
    val nama: String
) {
    var pendapatan: Int = 0
```

Kotlin

Solutions Case studies Docs API Community Teach Play

2.3.21 JVM Program arguments Copy link Share code Run

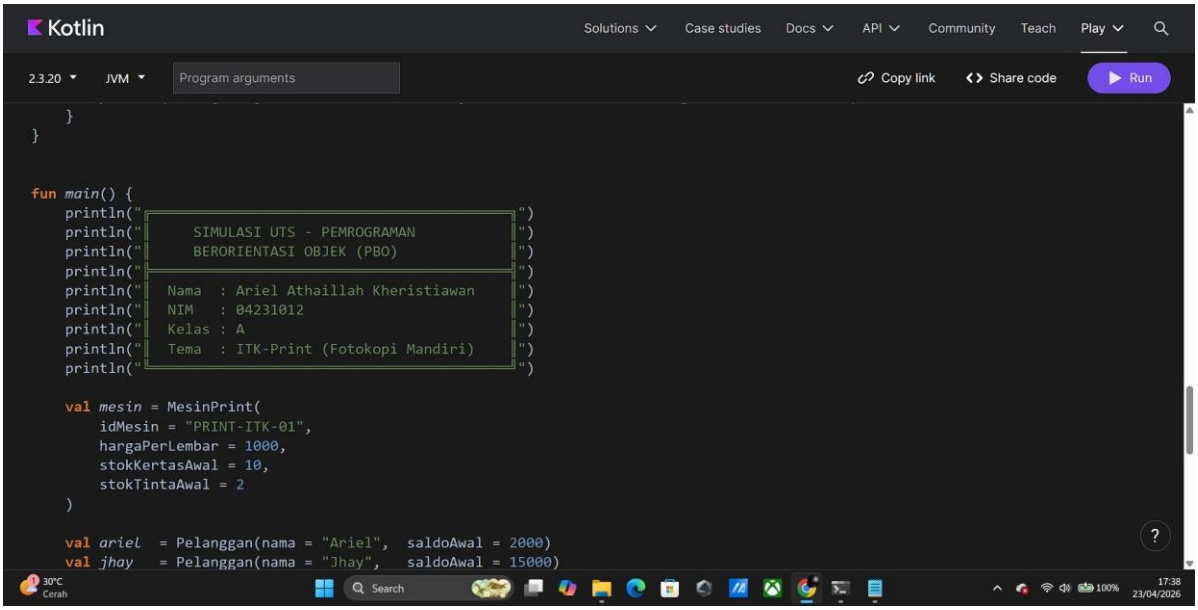
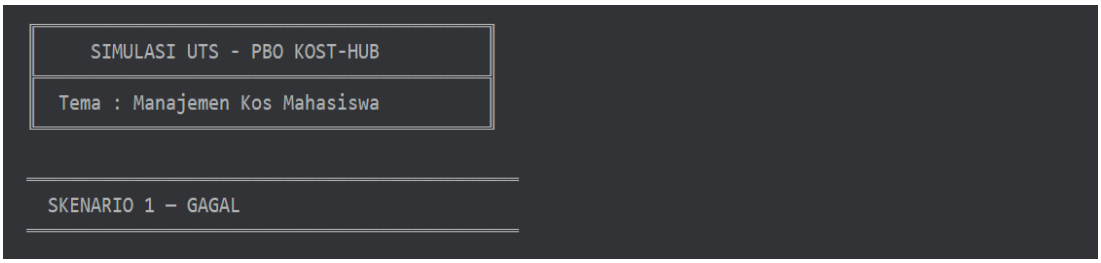
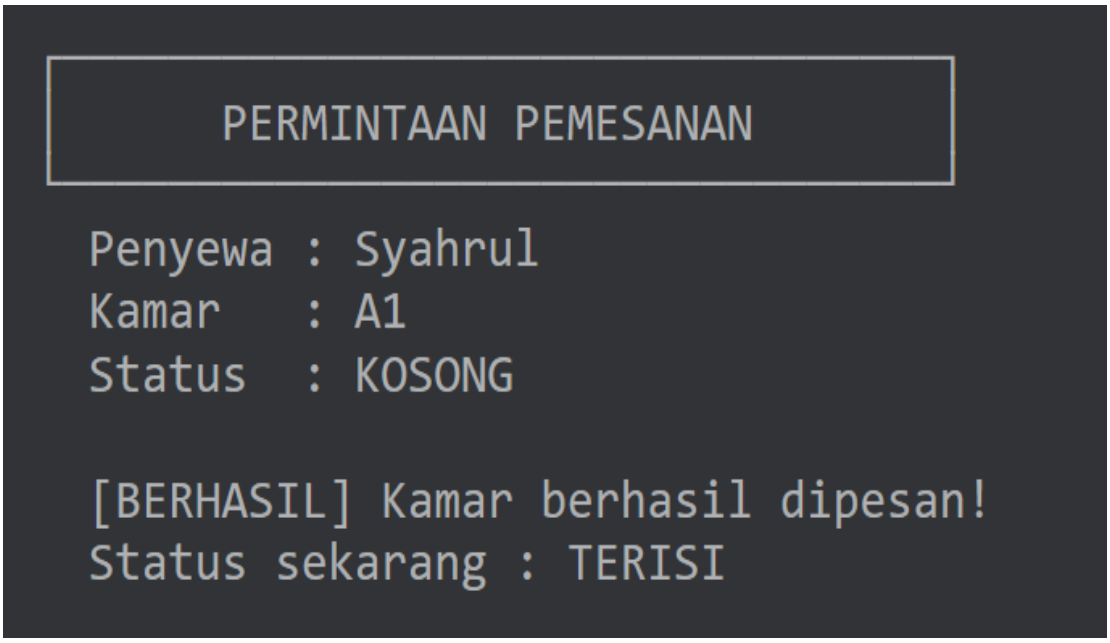
```kotlin
class BapakKos(
    val nama: String
) {
    var pendapatan: Int = 0
        private set

    fun tambahPendapatan(jumlah: Int) {
        pendapatan += jumlah
    }
}

class Kamar(
    val nomor: String,
    val harga: Int
) {
    var status: StatusKamar = StatusKamar.KOSONG
        private set

    fun pesan(): Boolean {
        return if (status == StatusKamar.KOSONG) {
            status = StatusKamar.TERISI
            true
        } else {
            false
        }
    }
}

class TransaksiKos {
```



- *Screenshot Output Codingan Kotlin*